

El algoritmo de Chaitin

Asignación de registros por coloreo de grafos

Equipo:	Daniela Nava Martínez · Max Uriel Sánchez Díaz
Voz en off:	Max Uriel Sánchez Díaz
Materia:	Traductores / Compiladores · Prof. Antonio López Jaimes
Duración:	Guión completo: aprox. 10 min. La tarea pide 5–7 min: si necesitas ajustarte, habla a buen ritmo o recorta.

Lee cada bloque mientras muestras la diapositiva correspondiente en la web. Los tiempos son una guía aproximada.

1. Portada

0:00 – 0:30

Hola, somos Daniela Nava Martínez y Max Uriel Sánchez Díaz. En este video vamos a explicar el algoritmo de Chaitin, uno de los métodos clásicos para la asignación de registros dentro de un compilador. Veremos primero qué problema resuelve y por qué es importante, luego cómo funciona paso a paso apoyándonos en diagramas, después dos ejemplos completos —uno sin derrame y otro con derrame— y, para terminar, nuestras conclusiones sobre cuándo conviene usarlo y si tendría sentido aplicarlo en nuestro propio traductor. Empecemos.

2. El problema

0:30 – 1:20

Para entender Chaitin, primero hay que entender el problema que resuelve. Cuando un compilador traduce nuestro programa, no trabaja directamente con los pocos registros del procesador: genera código intermedio usando una cantidad prácticamente ilimitada de registros virtuales, o temporales, que llamamos t_1 , t_2 , t_3 , y así sucesivamente. El problema es que el procesador real solo tiene un número muy pequeño de registros físicos: según la arquitectura, 8, 16 o 32. Entonces, en algún momento hay que decidir qué temporales se guardan en registros —que son rapidísimos— y cuáles se mandan a memoria, que es mucho más lenta. A eso se le llama asignación de registros, y es una de las últimas fases del back-end del compilador. La regla fundamental que hay que respetar es esta: si dos variables están vivas al mismo tiempo, es decir, si sus valores se necesitan a la vez, entonces no pueden compartir el mismo registro, porque una sobrescribiría a la otra. Todo el algoritmo gira en torno a esa restricción.

3. La idea clave

1:20 – 2:10

La gran idea de Chaitin, desarrollada por Gregory Chaitin y su equipo en los laboratorios de IBM a principios de los años ochenta, a partir de una sugerencia de John Cocke, fue darse cuenta de que este problema es exactamente el mismo que colorear un grafo. La analogía funciona así: cada variable del programa se convierte en un nodo del grafo. Cada vez que dos variables interfieren —o sea, viven al mismo tiempo y por tanto no pueden compartir registro— dibujamos una arista entre sus nodos. Y cada registro físico disponible lo representamos con un color distinto. Con esta traducción, el problema de asignar k registros se convierte en el problema de colorear el grafo usando k colores, con la condición de que dos nodos unidos por una arista nunca tengan el mismo color. Si logramos ese coloreo, tenemos una asignación válida de registros; y si no lo logramos con k colores, significa que no alcanzan los registros y habrá que mandar algo a memoria. Esta forma de ver el problema es la aportación central de Chaitin.

4. Motivación

2:10 – 3:00

Ahora bien, ¿por qué fue tan importante este enfoque?, ¿qué mejora respecto a lo que se hacía antes? En clase vimos la asignación local por descriptores de registros, ese método que trabaja instrucción por instrucción dentro de un bloque básico, decidiendo sobre la marcha qué registro usar. Es sencillo y rápido, pero es local: no ve el panorama completo de la función, desperdicia registros y termina metiendo muchos accesos a memoria entre bloques. Aquí conviene ser precisos: la verdadera aportación de Chaitin no fue simplemente hacer asignación global, porque ya existían esquemas globales anteriores, como los basados en prioridad. Lo realmente novedoso fue formular la asignación de registros como un problema de coloreo del grafo de interferencia; es decir, darle un modelo matemático claro, general y sistemático. Además, ese modelo habilita otras optimizaciones elegantes, como el coalescing, que elimina copias redundantes fusionando variables. Un detalle importante: colorear un grafo con tres o más colores es un problema NP-completo, así que no se conoce una forma eficiente de encontrar siempre la solución óptima. Por eso Chaitin no busca el óptimo, sino que usa una heurística muy ingeniosa que funciona muy bien en la práctica.

Antes de colorear necesitamos construir el grafo de interferencia, que es la entrada del algoritmo. Para ello se hace un análisis de vida, o liveness, que calcula, para cada variable, en qué punto del programa nace y en qué punto se usa por última vez; ese intervalo es su rango de vida. En la ilustración de la izquierda, cada barra de color representa el rango de vida de una variable. La regla es visual y muy intuitiva: cuando dos barras se solapan verticalmente, esas variables están vivas a la vez, así que interfieren, y por cada interferencia dibujamos una arista en el grafo de la derecha. Por ejemplo, aquí las variables *b* y *c* coinciden durante varias instrucciones, así que quedan conectadas. Hay una excepción importante, que suele preguntarse en compiladores: si dos variables solo coinciden en una instrucción de copia, del tipo *b* igual a *a*, no se pone arista entre ellas, aunque técnicamente coincidan en ese punto. ¿Por qué? Porque precisamente esa ausencia de arista es la que después permite fusionarlas con el coalescing y ahorrarnos la copia. Una vez construido, este grafo es lo que vamos a colorear.

El corazón del algoritmo es una idea sorprendentemente antigua: la heurística de Kempe, de 1879, que originalmente se propuso para el problema de los cuatro colores en mapas. La observación es la siguiente: si un nodo tiene menos de *k* vecinos, entonces siempre lo vamos a poder colorear, sin importar cómo queden coloreados los demás. La razón es simple: si tiene menos de *k* vecinos, esos vecinos usan como máximo *k* menos uno colores, así que siempre sobra al menos un color libre para él. Chaitin aprovecha esto en dos fases. La primera se llama Simplify, simplificar: recorremos el grafo y vamos quitando repetidamente cualquier nodo con grado menor que *k*, y lo apilamos. Lo interesante es que, al quitar un nodo, sus vecinos pierden una conexión, así que sus grados bajan, y eso normalmente permite quitar todavía más nodos en cadena. Cuando el grafo queda vacío, empieza la segunda fase, Select, seleccionar: vamos sacando los nodos de la pila en orden inverso y a cada uno le asignamos un color —un registro— que no estén usando sus vecinos ya coloreados. Y, gracias a Kempe, siempre habrá al menos uno disponible. Esta es la mecánica cuando el grafo sí se puede colorear.

Si juntamos todo, el algoritmo completo de Chaitin tiene varias fases encadenadas. Build construye el grafo de interferencia a partir del análisis de vida. Coalesce fusiona las variables unidas por una copia que no interfieren entre sí, para eliminar movimientos innecesarios; normalmente, después de fusionar se vuelve a reconstruir por si aparecen nuevas oportunidades. Luego se calculan los costos de derrame de cada variable, un número que nos dirá a quién conviene mandar a memoria si hace falta. Después viene Simplify, que apila los nodos fáciles como ya explicamos, y finalmente Select, que asigna los registros. Y aquí está la parte cíclica: si durante el proceso hubo que derramar alguna variable de verdad, se inserta en el programa el código para guardarla y cargarla desde memoria. Pero como ese código nuevo cambia los rangos de vida, hay que reconstruir el grafo desde cero y repetir todo. Se itera así hasta que ya no haya derrames, y en ese momento terminamos con una asignación válida.

8. Derrame (spilling)

Pero ¿qué pasa cuando nos atoramos? Es decir, ¿qué pasa si llegamos a un punto en el que ningún nodo tiene grado menor que k , todos tienen demasiados vecinos? En ese caso no podemos aplicar Kempe y hay que tomar una decisión difícil: derramar una variable, mandarla a memoria para liberar un registro. La pregunta clave es: ¿cuál elegimos? Chaitin propone una heurística muy razonable: elegir el nodo que minimice el cociente entre su costo de derrame y su grado. El costo de derrame estima cuántas instrucciones de carga y almacenamiento tendríamos que añadir si mandamos esa variable a memoria, y se pondera por la profundidad de los bucles, porque derramar algo que está dentro de un bucle que se ejecuta miles de veces es carísimo. El grado, por su parte, es cuántas otras variables estorba. Así, un nodo con bajo costo y alto grado es el candidato ideal: es barato de sacar y, al sacarlo, libera muchísimas interferencias de golpe. Vale la pena mencionar una mejora posterior muy conocida: el Chaitin original es pesimista, derrama en cuanto se atora; pero en 1994, Briggs, Cooper y Torczon propusieron el coloreo optimista, que en lugar de derramar de inmediato apila el nodo problemático y solo lo derrama si, al llegar a Select, de verdad no encuentra ningún color libre. Muchas veces sí lo encuentra, así que este enfoque colorea más grafos sin derramar.

9. Ejemplo 1 — sin derrame

6:15 – 7:10

Veamos todo esto en acción con nuestro primer ejemplo, en el que tenemos tres registros disponibles, es decir, k igual a tres. Observen el grafo que aparece en pantalla, con cinco variables: a , b , c , d y e . Empezamos con la fase Simplify. Los nodos d y e son los más fáciles: d solo tiene un vecino y e tiene dos, ambos con grado menor que tres, así que los quitamos y los ponemos en la pila. Fíjense en lo que ocurre al quitarlos: los nodos a , b y c , que formaban un triángulo, ahora bajan a grado dos, que también es menor que tres, así que también podemos apilarlos uno por uno. Con el grafo ya vacío, empieza la fase Select: vamos sacando los nodos de la pila en orden inverso y a cada uno le damos el registro más bajo que no usen sus vecinos. Uno tras otro, todos encuentran color: unos reciben $R1$, otros $R2$ y otros $R3$. El resultado final es que logramos colorear el grafo completo con solo tres registros y sin necesidad de derramar absolutamente nada. Este es el caso ideal. En la página, si quieren, pueden reproducir la animación para ver el proceso paso a paso.

10. Ejemplo 2 — con derrame

7:10 – 8:10

Ahora un ejemplo más interesante, donde las cosas no salen tan fáciles. Seguimos con tres registros, pero ahora el grafo es un $K4$: cuatro variables, a , b , c y d , donde todas interfieren con todas. Aquí hay un problema de fondo: un grafo $K4$ necesita cuatro colores por su propia estructura, pero nosotros solo tenemos tres registros. Veámoslo con el algoritmo. En Simplify, todos los nodos empiezan con grado tres, así que ninguno tiene grado menor que k , y nos atoramos de inmediato: toca derramar. Aplicamos la heurística de costo entre grado. Miren los cocientes: la variable c es la que tiene el menor valor, tres entre tres igual a uno, así que la elegimos como candidata a derrame y la sacamos del grafo. Al quitar c , los tres nodos restantes — a , b y d — forman un simple triángulo que sí es coloreable con tres colores, así que Simplify continúa sin problema. En Select, coloreamos a , b y d con $R1$, $R2$ y $R3$. Pero cuando llega el turno de c , resulta que sus tres vecinos ya ocuparon los tres registros disponibles, así que c no encuentra ningún color libre: se confirma como derrame real y su valor vivirá en memoria. Como hubo un derrame, se inserta el código de carga y almacenamiento para c y se reconstruye el grafo; ese nuevo grafo, con los rangos de c ya partidos, ahora sí es coloreable. Con esto se ve claramente cómo Chaitin maneja los casos difíciles.

11. Comparación

8:10 – 8:50

Pongamos a Chaitin en contexto comparándolo con otros algoritmos. La asignación local por descriptores, la que vimos en clase, es muy rápida pero de baja calidad y solo trabaja de forma local. Chaitin, en cambio, es global y produce código de alta calidad, aunque es más lento, porque construir y manipular el grafo de interferencia puede ser costoso, incluso cuadrático en el peor caso. Briggs mejora a Chaitin con el coloreo optimista que ya mencionamos, logrando menos derrames sin perder calidad. Y, finalmente, está el Linear Scan, propuesto en 1999, que asigna registros recorriendo los intervalos de vida casi en tiempo lineal: es muchísimo más rápido, aunque su calidad es algo menor. Por eso el Linear Scan se usa típicamente en compiladores JIT, los que compilan mientras el programa se ejecuta, donde el tiempo de compilación es crítico; mientras que el coloreo de grafos se asocia más con compiladores optimizadores, que compilan con calma buscando el mejor código posible.

12. Conclusiones

8:50 – 9:30

Llegamos a nuestras conclusiones. ¿Cuándo conviene usar Chaitin? Conviene cuando hay más variables vivas que registros disponibles y buscamos código de alta calidad, algo típico en un compilador optimizador serio. ¿Es más complejo, pero vale la pena? Sí: es más complejo que la asignación local, porque exige implementar el análisis de vida y construir el grafo, pero a cambio reduce mucho los accesos a memoria y elimina copias, lo que mejora bastante el rendimiento del código generado. ¿Hay algoritmos mejores? Depende de qué estemos optimizando: si nos importa la calidad, Briggs mejora a Chaitin; George y Appel mejoraron el coalescing; y si lo que importa es la velocidad de compilación, el Linear Scan gana. No hay un único ganador: depende del contexto. Y quizá lo más elegante de todo el enfoque de Chaitin es que toma un problema teóricamente muy difícil, NP-completo, y lo resuelve con una heurística simple, comprensible y muy efectiva.

13. ¿En nuestro traductor?

9:30 – 10:00

¿Y usaríamos Chaitin en nuestro propio traductor? Seamos honestos: nuestro traductor actualmente hace asignación local por descriptores, tal como vimos en clase, y para el alcance del curso eso es más que suficiente. Usaríamos Chaitin si diéramos el siguiente paso: generar código de máquina real, con un número fijo de registros, y quisiéramos optimizar de verdad el uso del procesador. El costo de adoptarlo sería agregar un análisis de vida global y toda la construcción y el coloreo del grafo de interferencia, que es bastante más ingeniería. Así que, en resumen, no es necesario para nuestro proyecto actual, pero sería la evolución natural si algún día convertimos nuestro traductor en un back-end optimizador de verdad.

En resumen: el algoritmo de Chaitin modela la asignación de registros como un problema de coloreo de grafos, usa la heurística de Kempe para simplificar el grafo y, cuando no alcanzan los registros, derrama a memoria la variable más conveniente. A pesar de tener ya varias décadas, sigue siendo la base conceptual de la asignación de registros moderna. Y con esto terminamos. Muchas gracias por su atención.

Referencias

- Chaitin et al. (1981). Register Allocation via Coloring. Computer Languages.
- Chaitin, G. (1982). Register Allocation & Spilling via Graph Coloring. SIGPLAN.
- Briggs, Cooper & Torczon (1994). Improvements to Graph Coloring Register Allocation.
- Poletto & Sarkar (1999). Linear Scan Register Allocation.
- Appel, A. Modern Compiler Implementation, cap. 11.